Problem Statement : Write a program to implement Huffman Encoding using a greedy strategy.

Code:

```
import java.util.PriorityQueue;


class Node {
    char data;
    int frequency;
    Node left, right;

    Node(char data, int frequency) {
        this.data = data;
        this.frequency = frequency;
        left = right = null;
    }
}


class Compare implements java.util.Comparator<Node> {
    public int compare(Node a, Node b) {
        return a.frequency - b.frequency;
    }
}

public class HuffmanEncoding {


    static void generateCodes(Node root, String code) {

        if (root == null)
            return;


        if (root.left == null && root.right == null) {
            System.out.println(root.data + " : " + code);
            return;
        }

        generateCodes(root.left, code + "0");
        generateCodes(root.right, code + "1");
    }


    static void huffmanEncoding(char[] data, int[] frequency) {

        PriorityQueue<Node> pq = new PriorityQueue<>(new Compare());
```

```
        // Insert all characters into priority queue
        for (int i = 0; i < data.length; i++) {
            pq.add(new Node(data[i], frequency[i]));
        }


        while (pq.size() > 1) {

            Node left = pq.poll();
            Node right = pq.poll();

            Node newNode = new Node(
                '\0',
                left.frequency + right.frequency
            );

            newNode.left = left;
            newNode.right = right;

            pq.add(newNode);
        }


        Node root = pq.poll();

        System.out.println("Huffman Codes:");
        generateCodes(root, "");
    }

    public static void main(String[] args) {

        char[] data = {'A', 'B', 'C', 'D', 'E', 'F'};

        int[] frequency = {5, 9, 12, 13, 16, 45};

        huffmanEncoding(data, frequency);
    }
}
```

Output:

```
comp@comp-HP-ProDesk-600-G3-SFF:~/Aniket$ java HuffmanEncoding
Huffman Codes:
F : 0
C : 100
D : 101
A : 1100
B : 1101
E : 111
```